IC-272 — Introduction to Machine Learning

Lab 7 — Logistic Regression

Indian Institute of Technology Mandi

Total marks: 100 — Duration: 2 hours

Marks: 20 for code, 80 for the viva. There are three tasks. Your TA takes a short viva on each task *as soon as you finish it* — do not save them all for the end.

There is also a homework section at the end. It will not be vivaed and carries no marks today. It is examinable in the final lab exam.

You will reuse `design_matrix` and `fit_normal_equation` from Lab 4, `standardise` from Lab 5, and your confusion-matrix and precision/recall/F_1 functions from Lab 1. Copy them into a cell at the top.

Rules

- Permitted: `numpy`, `pandas`, `matplotlib`. `scikit-learn` **is not permitted** — a task that calls it scores zero for that task.
- Logistic regression has no closed-form solution. You fit it by gradient ascent, as in the lecture; there is no library shortcut allowed.

Setting up — type this first

`ic272_lab7_loans.csv` holds 500 loan applicants. `income_k` is monthly income in thousands of rupees, `utilisation` is the fraction of their credit limit currently in use, and `defaulted` is the label: 1 if they failed to repay.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

df = pd.read_csv("ic272_lab7_loans.csv")
features = ["income_k", "utilisation"]
X = df[features].to_numpy(float)
y = df["defaulted"].to_numpy(int)

idx = np.random.default_rng(0).permutation(len(y))
te, tr = idx[:100], idx[100:]
X_train, y_train = X[tr], y[tr]
X_test,  y_test  = X[te], y[te]

print("train:", X_train.shape, " test:", X_test.shape)   # (400, 2)  (100, 2)
```

Use `seed = 0` exactly as written. About 30% of the test applicants default, so predicting "repaid" for everyone already scores 0.70 accuracy. That is the number to beat, and it is why accuracy alone will not be enough today.

Why this lab exists — read this, then run it

You already have a model that fits a line to data: linear regression from Lab 4. Why not use it here, with the labels 0 and 1 as the target, and read the output as a probability?

```
theta_lin = fit_normal_equation(X_train, y_train.astype(float))
pred = design_matrix(X) @ theta_lin

print("smallest prediction:", pred.min().round(3))
print("largest prediction: ", pred.max().round(3))
print("predictions below 0:", (pred < 0).sum())
```

You will find 62 applicants with a *negative* predicted value — as low as -0.556. A probability cannot be negative. Linear regression has no idea its output is supposed to be a probability; it will happily go below 0 or above 1 wherever the straight line takes it.

The fix: squash the line through the sigmoid

Keep the linear part, $z = \theta^\top x$, but pass it through

$$\sigma(z) = \frac{1}{1 + e^{-z}}.$$

For very negative z this is close to 0, for very positive z close to 1, and at $z = 0$ it is exactly $\frac{1}{2}$. So $h_\theta(x) = \sigma(\theta^\top x)$ is always a valid probability, and we read it as $P(y = 1 \mid x)$.

There is now no formula for θ like the normal equation. Instead we choose the θ that makes the observed labels most probable — maximum likelihood, as in the lecture. Working with the logarithm turns a product of m probabilities into a sum:

$$\ell(\theta) = \sum_{i=1}^{m} \Big[y^{(i)} \log h_\theta(x^{(i)}) + \big(1 - y^{(i)}\big) \log \big(1 - h_\theta(x^{(i)})\big) \Big].$$

Each term rewards a confident correct prediction and punishes a confident wrong one. We climb this function with *gradient ascent*. In matrix form, for the whole training set at once, the update from the slides becomes

$$\theta \;\leftarrow\; \theta + \frac{\alpha}{m} A^\top \big(y - \sigma(A\theta)\big),$$

where A is the design matrix from Lab 4. Note the plus sign: we are maximising.

As in Labs 5 and 6, **standardise the features first**. Income runs into the hundreds and utilisation sits between 0 and 1; on the raw columns gradient ascent either crawls or oscillates.

Task 1 — Sigmoid and log-likelihood [code 6 — viva 24]

(a) Write `sigmoid(z)`, vectorised so it works on a whole array at once. **[2]**

(b) Write `log_likelihood(X, y, theta)` computing $\ell(\theta)$ above, where `X` has *no* column of ones — build A inside the function with `design_matrix`. **[3]**

(c) Standardise the features, then evaluate `log_likelihood` on the training set at $\theta = \mathbf{0}$. Print the total and the value per training example. **[1]**

$\log 0$ is $-\infty$. Add a tiny constant such as 10^{-12} inside each `np.log` so that a prediction of exactly 0 or 1 does not break the sum.

Check as you go. At $\theta = \mathbf{0}$ every prediction is $\sigma(0)$. Work out by hand what the per-example value must therefore be, before you run (c). If your code disagrees with your hand calculation, one of them is wrong — find out which before calling your TA.

Task 2 — Fitting by gradient ascent [code 8 — viva 32]

(a) Write `fit_logistic(X, y, lr=0.1, n_iters=2000)` that starts from $\theta = \mathbf{0}$, applies the update above `n_iters` times, and returns the final θ together with the list of log-likelihood values, one per iteration. **[5]**

(b) Fit it on the standardised training set and print θ. **[1]**

(c) Plot the log-likelihood against iteration number. **[2]**

This is your Lab 4 gradient-descent loop with two changes: the prediction goes through `sigmoid`, and you add the step instead of subtracting it. Write it so that you can say exactly which line changed and why.

With a small enough learning rate the log-likelihood must **never decrease**. If your curve goes down anywhere, check your sign.

Task 3 — Thresholds and evaluation [code 6 — viva 24]

The model outputs a probability. To get a yes/no decision you must pick a threshold.

(a) Compute the predicted probability of default for every *test* applicant. Remember the test set is standardised with the *training* μ and σ. **[2]**

(b) Convert to labels with a threshold of 0.5, and report accuracy, precision, recall and F_1 using your own Lab 1 functions. **[2]**

(c) Repeat (b) with a threshold of 0.3. **[2]**

Compare both against the 0.70 baseline. Then look at what moved between the two thresholds: one metric rises sharply, another falls. Be ready to say which threshold you would use if you ran the bank, and why the answer depends on what a missed default costs compared with a wrongly refused loan.

Homework — not vivaed today, examinable in the final lab exam

Complete this homework before the next lab. The ROC and AUC functions from H1 will be used again in the coming labs, which will call them by the names given below and will not ask you to write them a second time. The other homework items are also built on in later labs. If you skip them now, you will be writing them under time pressure later.

H1. ROC curve and AUC. Write `roc_curve_scratch(y_true, p)` returning the arrays of FPR and TPR, and `auc_trapezoid(fpr, tpr)` returning the area. Keep exactly these names. Task 3 tried two thresholds; now try all of them. Sort the test probabilities, use each one in turn as the threshold, and at each compute

$$\mathrm{TPR} = \frac{TP}{TP + FN} \ \text{(this is recall)}, \qquad \mathrm{FPR} = \frac{FP}{FP + TN}.$$

Plot TPR against FPR, with the diagonal for reference. Then compute the area under the curve with the trapezoid rule,

$$\mathrm{AUC} = \sum_i \left(\mathrm{FPR}_{i+1} - \mathrm{FPR}_i\right) \frac{\mathrm{TPR}_{i+1} + \mathrm{TPR}_i}{2},$$

with the points in increasing order of FPR. This is the Lab 3 threshold sweep again, with real probabilities this time. Say what an AUC of 0.5 and of 1.0 would mean.

H2. Draw the decision boundary. The boundary is where $p = 0.5$, which is where $\theta^\top x = 0$ — a straight line. Solve that equation for `utilisation` in terms of `income_k` (remember θ was fitted on standardised features, so undo the standardisation) and draw the line over a scatter plot of the data, coloured by class.

H3. Where logistic regression fails. The lecture's one stated limitation is that the decision boundary is linear. Test it. Fit your `fit_logistic` on the Lab 6 ring data, `ic272_lab6_vibration.csv`, using `vib_x` and `vib_y` with the same split as Lab 6, and compare its test accuracy with your Lab 6 k-NN result. Then add one extra feature, $r^2 = \texttt{vib_x}^2 + \texttt{vib_y}^2$, refit, and explain why the accuracy changes so much.

H4. Why standardise? Run `fit_logistic` on the *raw* loan features with `lr=0.001` and with `lr=0.01`, and plot both log-likelihood curves. Describe what each does and connect it to what you saw in Lab 4.

Submission

- Call your TA as you finish each task. All three vivas must be done before you leave.
- Keep the notebook; the homework goes in the same file.